CTCPU and Hunter: The Road So Far

I compiled this journal from my own words and memo records. Most were spontaneous notes I made at the time. I have retold instructions given during conversations and fragmentary statements in the first person, preserving their meaning; these passages are not verbatim quotations. Passages left in quotation marks are English translations of my recorded Turkish words. I keep the original records in my archive.

---

1. One-page overview

This is not a proof notebook. It records where my thinking went, step by step: something caught my attention, I asked myself a question, tried something, and the result took me somewhere else. The dates and times show the order of that journey; short mathematical arguments are in section 3, and finite computational checks are in section 4. Passing a test is not, by itself, a general proof.

The machine was initially called CT-CPU. When I decided to rewrite it on 30 August 2026, it became Hunter; in this document, the two names refer to different periods of the same project. CT-CPU is not presented as a completed processor; Hunter continues that work.

The shadow is a second representation in which I keep a number's prime factors and their exponents. Multiplication becomes addition of exponents, and division becomes subtraction of exponents. RS means shadow register; `RS1[e3=7]` means that the exponent of the prime 3 in the first shadow register is 7. In LOG 020, I proposed using this notation to split 3^11 between two shadow registers.

For a full year I have been working on two things: designing my own processor and playing with Mersenne numbers. I will not say that I "discovered" everything here. Most of it began with me asking myself "could it work this way?" and then trying it. Some things held. Some were wrong. I later learned that some had been known for centuries. I am writing them all down in my own words, in order.

[D]101 is a notation for a number whose binary representation ends in 101, with D standing for the part before it; the number is therefore 8D + 5. For integers p ≥ 3 and k ≥ 0, every number (3+8k)·M_p ends this way.

"I think Pascal may have built my processor before I did, and the answer was already there."

Here is a brief outline of which idea prompted which. All dates are in 2026.

Date	What happened	What it prompted
21 April	CT-CPU description: colour-labelled xbit and octal; Mersenne exponents R and Y modulo 3	The octal language; the conceptual root of 1-3-7
23 April	LOG mode: multiplication as index addition	The principle of the shadow
13 May	One xbit field selects eight types	Today's TYPE cell
8 June	From wanting Doom to CT-CPU; 1 xbit = 3 bits = 1 colour, "Decision: FINAL"	The whole machine language
9 June	Shadow: multiplication adds exponents, division subtracts them	Resolver, rough, lazy shadow
16-19 June	Mersenne xbit pattern: W, RW, YW	Octal Mersenne patterns
20-22 June	One central shadow resolver; digit sum modulo 7	Digit flow

Date	What happened	What it prompted
18 July	Manifesto: two faces, the 3.75-bit limit, God does not play dice	Acceptance criteria
26 July	M_p = leading digit + 7s; q = 2kp + 1 recorded	Sieve gates
Between 26 July and 2 August	Counting machine; FOLD = mod M_p (from a later compilation)	Screw and turns
29 July	The engine follows the representation; Mersenne as test bed; 8 opcodes	Hunter's simple ISA
Night of 2 August	1, 3, 7 replace R, Y, W	The 1-3-7 universe
5 August	The 1-3-7 tally universe	Three leading digits, package
21 August	"Does this exist elsewhere?"; matching Euler and Fermat	The habit of separating known results
30 August	Horner chain "235"; the overflowing-digit question	C/T/A, [D]101
31 August	"121 = 11 times 11"; shadow does not help LL; p = 3q + r; base 8 confirmed	Pascal; package
Night of 1 September	1331 in every base; carry-free universe; "the most important number is 3"	The 11 formula, Base^0
2 September	Pascal night; threshold C(n, ⌊n/2⌋) + 1	The ladder
2-3 September	Theory compilation: C/T/A, three leading digits, factor law, screw, box	Tests
Morning of 7 September	Signature filter failed at the 51st and 52nd exponents; packed sieve; 11 scale	Sieve round
15 September	Reversed Mersenne P; [D]101 family; the name Base^0	Specificity test
25 September	From Scott's book to HunterCPU	The pure CPU book (1 October)
Night of 1 October	Binary folding with 13, 31, 127; objection to "p >= 2"	Open intuitions
10 October	182 theories, tested list; proof of [D]101	This document
11-12 October	Ladder step: a_(n+1) = a_n + kom_n, flip-flop	Adder circuit
12 October	Chain in base 3; period five at 121	The trace of 11
13-14 October	Power of 2 in a Mersenne base: clean Pascal row, 52 exponents	Where I arrived

How I work

I am still not very comfortable with Python; I am still trying to grasp it. I am not good at C either; sometimes even that feels high-level to me. I realised rather late that I needed the GMP library for large numbers; of course I used artificial intelligence for help. Python and an LLM do not feel very different to me: with either, you write something and it handles it.

I feel closer to gears, wheels and things like torque. I am a physicist, but classical mechanics feels more quantised to me; quantum physics feels more analogue, more delicate and uncertain.

I like precise, step-by-step progress. I often cannot fully understand what people mean, either; because I like certainty and clarity, anything high-level comes with question marks for me. As a child I was enthusiastically learning QBasic; perhaps that is where this fixation on precise statements began. Instructions such as “if this is this, then that is that; now go here” have always felt simpler. Perhaps because they carry less responsibility.

The irony is that I am comfortable with 1 and 0, yet I am trying to build an algebra between 0 and 7. That is why the main tests in this document were written in C.

2. ADHD journal

Each entry follows the same pattern: what I sensed, what I asked myself, what I looked at and tried, the outcome, where it led, and what I learned. Where I did not record a field, I omitted it rather than leaving it blank. The original records of the Mersenne branch from June to August are not on this computer; that period survives here only through traces in later compilations.

My earliest available written record is from April 2026. The months before that are not in this journal; I do not have their written records.

LOG 048–061 are not included in this compilation.

LOG 001 · 21 April 2026 · Colourful coprocessor

Research and test: I described CT-CPU (now Hunter) as a coprocessor with colour-labelled registers and octal representation. I classified Mersenne prime exponents modulo 3 and marked them with R and Y; I arranged Mersenne numbers in an octal table, with a colour letter for each digit. I counted transitions between consecutive primes modulo 3. Lemke Oliver and Soundararajan published the last-digit bias in 2016; I learned of the paper later. This motivated me: I should educate myself further, because I had reached this point with pen and paper. Something else motivates me too: it would actually be absurd to think that nobody had found the things I was thinking about before me. America was already there before I discovered it. Reaching this point so quickly with pen and paper was, separately, very motivating indeed. I really understood what I learned while studying physics. Being successful at something does not mean we understand it; that was the main lesson I took from this.

Outcome: KNOWN IN THE LITERATURE. My count over 52 Mersenne exponents showed no bias; as this was statistical, I did not treat it as a verdict.

Led to: → LOG 004 (xbit), → LOG 006 (Mersenne xbit pattern)

LOG 002 · 23 April 2026 · LOG mode

Research and test: I wrote a “LOG mode” that performs multiplication by adding logarithmic indices and division by subtracting them.

Outcome: OPEN. A single-xbit multiplication and division test was written the next day; there is no run output.

Led to: → LOG 005 (shadow)

LOG 003 · 13 May 2026 · Eight types

Research and test: I decided that one xbit field would select eight types.

Led to: → today’s TYPE cell (section 3)

LOG 004 · 8 June 2026 · From Doom to a processor: xbit

Research and test: I wanted to write Doom in assembly. First I needed to understand how a processor works, then design my own. Computing with light: three light channels give eight colours, and eight colours make one octal digit. 1 xbit = 3 bits = 1 octal digit = 1 colour. I wrote “Decision: FINAL” in the journal.

Led to: → LOG 005; the whole machine language starts here

LOG 005 · 9 June 2026 · Shadow

Research and test: I rebuilt the LOG-mode principle using prime exponents instead of logarithms. Beside each number's positional representation there is a shadow; multiplication in the shadow adds exponents, and division subtracts them. On 13 June I searched earlier work: Gödel numbering (1931), the logarithmic number system (1971).

Outcome: KNOWN IN THE LITERATURE. The shadow rule is the fundamental theorem of arithmetic (T13_shadow_vector).

What I learned: I realised I had been doing quite a lot of this while working as a bartender. In June I had written to myself: "I find it myself, then see it in the literature (convergent insight)."

Led to: → LOG 007, → LOG 008

LOG 006 · 16-19 June 2026 · Mersenne xbit pattern

Research and test: I wrote 2^p − 1 as W, RW or YW according to the class of p modulo 3.

Outcome: HOLDS. In July, 10 examples between p = 5..127 produced no exceptions; the general form is T05_137_octal_head.

Led to: → LOG 010, → LOG 013

LOG 007 · 20-22 June 2026 · Shadow resolver and digit flow

Research and test: I built one central shadow resolver. I wrote that the remainder modulo 7 is given by the sum of the octal digits. I called this division-free remainder walk digit flow.

Outcome: KNOWN IN THE LITERATURE. This is Horner's 1819 scheme; in my own note I had said, "This seems like some Horner-like software" (T12_digit_flow).

Led to: → LOG 011

LOG 008 · 18 July 2026 · Manifesto

Research and test: I wrote my manifesto: each number lives with two faces, the shadow's 3.75-bit limit, and the later addition "God does not play dice."

Led to: → section 7, manifesto; acceptance criteria for the machine

LOG 009 · 18-19 July 2026 · Keep the edge

Research and test: I wrote down the idea of keeping the edge instead of calculating the square root.

Outcome: KNOWN IN THE LITERATURE. It resembles Wildberger's rational trigonometry; I saw this later.

LOG 010 · 26 July 2026 · Leading digit, 7s, factor law

Research and test: I wrote M_p as a leading digit followed by 7s. A task file records q = 2kp + 1.

Outcome: KNOWN IN THE LITERATURE. Fermat knew the divisor form q = 2kp + 1 in 1640; Euler added q ≡ ±1 (mod 8) in 1750 (T03_factor_law). I found the source when I investigated.

Led to: → LOG 031

LOG 011 · 26 July-2 August 2026 · Counting machine and FOLD

Intuition: I had not become comfortable with the word mod. Dividing a number and writing the remainder felt unfamiliar; I wanted to understand the same operation through a different movement.

Research and test: Since 2^p ≡ 1 (mod 2^p − 1), I added the high bits to the low bits to find the remainder without division. I called this KATLA (FOLD); I called it a turn of a counting machine.

Outcome: KNOWN IN THE LITERATURE. The known end-around-carry method for a Mersenne modulus; the arithmetic beneath the screw is multiplicative order. It was verified over 45,056 identities at the end of September.

Led to: → LOG 019, → LOG 078

What I learned: What I called “folding” was modular arithmetic itself.

LOG 012 · 29 July 2026 · Representation decision

Research and test: My decision: the engine follows the representation, not the mathematics; Mersenne is the test bed. I reduced the instruction set to eight opcodes.

Led to: → Hunter’s simple instruction set (section 3)

LOG 013 · Night of 2 August 2026 · 1, 3, 7 replace R, Y, W

Intuition: I thought I could keep the repetition count instead of writing 734 copies of 7. When that number also became very large, I considered using a representation of the form 2^p.

Research and test: I began writing 1, 3, 7 instead of the colour letters.

Led to: → LOG 014

LOG 014 · 5 August 2026 · The 1-3-7 tally universe

Intuition: “Just because the 21st apple went into the 2nd box, the 2nd box does not mean ‘20’; there are 20+1 apples.”

Research and test: I treated 1, 3, 7 as marks; they count quantities. I called this a number universe.

Led to: → LOG 028, → LOG 031

LOG 015 · 21 August 2026 · Does this exist elsewhere?

Question: I wondered whether there were other projects similar to mine.

Research and test: The resulting research showed that four of my Mersenne observations were known to Euler and Fermat.

Outcome: KNOWN IN THE LITERATURE. If p ≡ 3 (mod 4) and 2p + 1 is prime, then 2p + 1 divides M_p: Euler’s result (T04_euler_2p_plus1).

What I learned: I made a habit of separating known results.

LOG 016 · 25 August 2026 · Division from the right

Research and test: I recorded a procedure that divides a number starting at its lowest digit.

Outcome: KNOWN IN THE LITERATURE. Close to Montgomery’s 1985 method for modular multiplication.

LOG 017 · 30 August 2026 · Overflowing digit

Question: When adding a number that is 10^7 to another with a factor of 10^3, I asked whether I could factor out 10^3 and work through 10^4 and 10^0.

Research and test: I walked the Horner chain through “235”.

Led to: → LOG 025, → LOG 036

LOG 018 · 31 August 2026 · Two branches meet

Intuition: I thought my Mersenne work and processor idea were meeting at the same point.

Research and test: I brought the Mersenne work and Hunter together. In my head I was thinking in decimal and writing in octal. I tried not to confuse the base in which I calculated with the base in which I wrote the result. My handwritten list gave the octal forms for exponents from 1 to 12: 1, 3, 7, 17, 37, 77, 177, 377, 777, 1777, 3777, 7777.

Led to: → LOG 019, → LOG 028

LOG 019 · 31 August 2026, 21:54 · Shadow does not help Lucas-Lehmer

Research and test: I compared bit-level folding and octal-digit folding for the Lucas-Lehmer remainder. Looking for a way out, I split the exponent as p = 3q + r. My decision: finish the processor first, LL later.

Outcome: FAILS. Bit folding was correct in all 500 out of 500 trials; octal-digit folding succeeded in 22 out of 500 for p = 7, and in none for p = 13, 17, 31. 8^k − 1 equals M_p only when p is a multiple of 3. Record title: “Octal and shadow do not help LL.”

Led to: → LOG 031 (the decomposition p = 3q + r became the basis of the package)

LOG 020 · 31 August 2026 · 3^11 and 121

Intuition: I was struck by 2·5·8+1 being written as 121 in octal; I connected it to 11 times 11. 2·5 was 10 in decimal; I read 10·8+1 as 10 and 1, like eleven. From (2·5·8+1)·3^7 I reached 121x3^7. I could not yet prove the relationship between 121 and the square; at this stage it was only an intuition.

Research and test: I reduced 3^11 stage by stage in the shadow: 8v + a = 81, v = 10, that is, 2 × 5. I proposed splitting 3^11 into two shadow rows, “RS1 [e3=7] RS2 [e3=1]”. RS means shadow register. A cell holds at most 7; therefore 3^11 splits as 3^7 · 81 and RS1[e3=7]. The remaining 81 is 121_8 in octal. Subtracting 1 gives 120_8, and dividing by 8 leaves 12_8. In octal, $12_8 = 2 \cdot 5$ (not $2^2 \cdot 3$, which is 14_8). Thus the correct second row is RS2[e2=1, e5=1 | a=1]:

```
rs1 = [e3 = 7]                  3^7 = 4213₈
rs2 = [e2 = 1, e5 = 1 | a = 1]  12₈ = 2·5,  a = 1  →  121₈
(12₈ · 10₈ + 1) · 4213₈ = 121₈ · 4213₈ = 531773₈ = 3^11   ✓
```

This was exactly what reading 2·5·8+1 as 121 amounted to: $2 \cdot 5 \cdot 10_8 + 1 = 121_8$.

Outcome: FAILS. 3^7 × 24 gave 52488, not 177147. But the mistake showed me 121.

Led to: → LOG 021, → LOG 022

LOG 021 · 31 August 2026 · Squares and base 11

Intuition: Among representations ranging from 121_2 to 121_16, I was looking at numbers such as 9, 64, 81, 121, 289. They felt like the primes of the number universe I was trying to build.

Question: I considered whether base 11 would be more suitable and whether I could build a parallel number universe in base 11 for Hunter.

Research and test: Why is 11 special? I had “because 11 is the smallest beautifully symmetric prime. The answer is around here.” When I saw that octal 11 is 9, I corrected myself: “I said the smallest symmetric one. There is no other symmetric one anyway, because it is a unicorn.” My decision: “base 8 approved.”

Outcome: FAILS. I had thought squares were palindromic in octal; only 6.5 percent of the squares from 1 to 200 are palindromes.

Led to: → LOG 022

LOG 022 · Night of 1 September 2026 · 1331 in every base

Intuition: I looked at the cube of 11 in base 10 being written as 1331, the cube of 12 in base 11 as 1331, and the cube of 13 in base 12 again as 1331. I thought Pascal might have built my processor before I did, and that the answer I sought was already there.

Research and test: When I increased the base by one, I increased the number by one as well; the written form stayed the same. In base b, (b + 1)^3 = 1331.

Outcome: KNOWN IN THE LITERATURE. Powers of 11 are Pascal rows; the binomial theorem (T07_pascal_11).

Led to: → LOG 023, → LOG 024, → section 3, threshold law

What I learned: I later named this idea “11 is a formula.”

LOG 023 · Night of 1 September 2026 · A carry-free universe

Intuition: After the fragmentary expression 121 x 11 = 11 in my note, I was trying to explain the calculation through 121x10 + 121. The first expression was not a correct equality, but my unfinished note at the time. I then wrote 14641*11 = 14641x10 + 14641 and placed the coefficients side by side as 15101051 without propagating carries. I thought of this as a carry-free number universe; it was not the same as ordinary decimal digit notation.

Question: I wondered whether I could read the divisibility rule for 11 through 9 in octal, and what the Horner expansion would show here. I also asked whether all binomial terms were multiples of 11.

Research and test: I read 11 as b + 1 in every base. Multiplying by 11 means multiplying by 10 and adding 10 percent; it looks this way only in decimal.

Outcome: KNOWN IN THE LITERATURE. 11_b = b + 1 is the definition of the notation (T09_11_base_value).

Led to: → LOG 076, → LOG 036

LOG 024 · 2 September 2026, 04:07 · Ladder and Pascal night

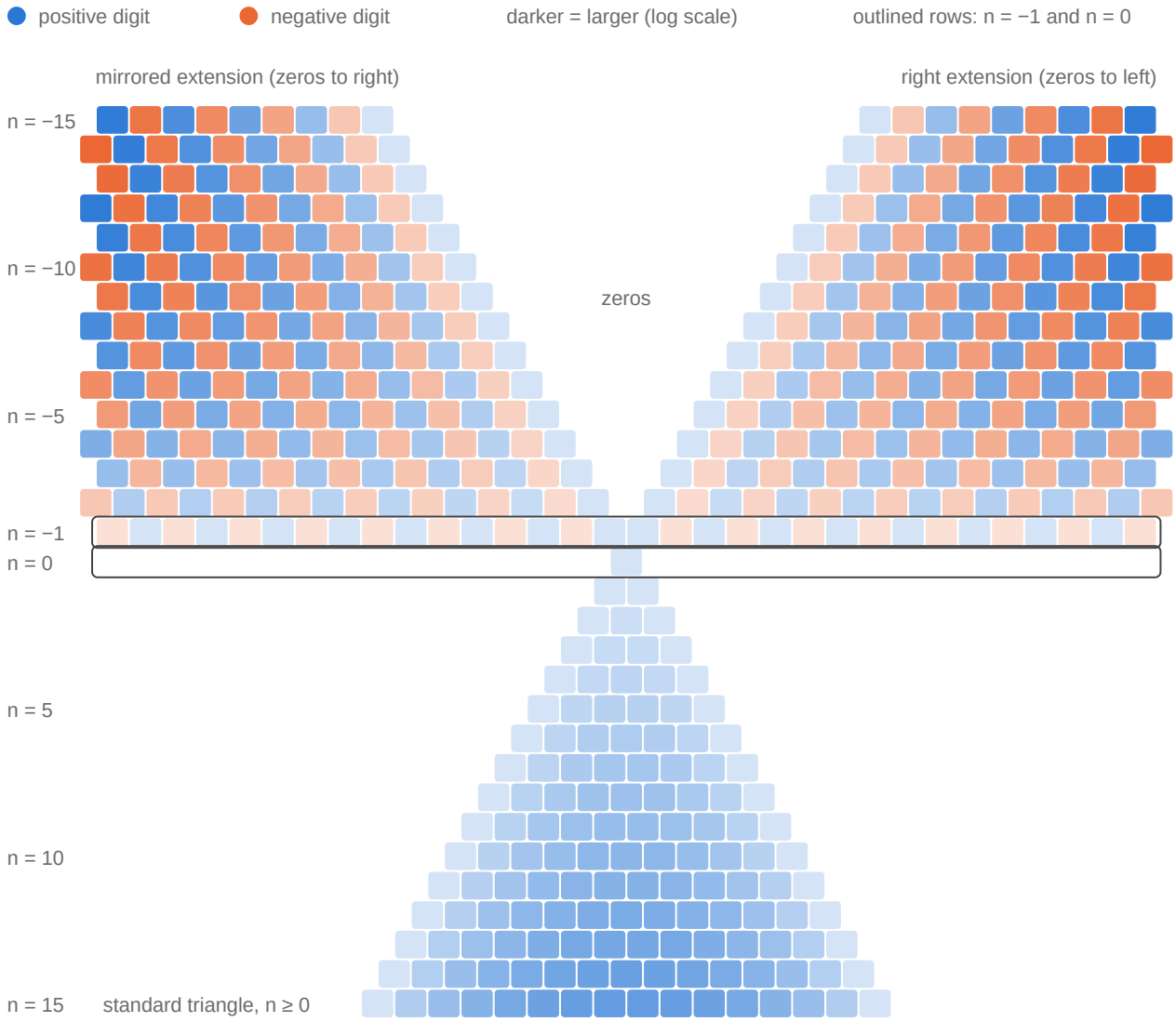
Intuition: When representations such as 1010001 appeared, I thought the palindrome was shifting. I tried to understand 11 becoming 12 at that point in terms of the number changing, rather than the base.

Question: While looking at the Pascal expansion, I asked myself why I had not increased the base through 11, 12, 13, 14 earlier.

Research and test: I looked at when carrying starts. The central coefficient, not n, determines the breaking point.

Outcome: HOLDS. The smallest base for writing 11^n without carries is C(n, ⌊n/2⌋) + 1 (T08_threshold_law). Pascal night entered the Hunter archive at this time.

Pascigon



The wings are separate continuations. Negative rows are infinite; the sides stop at x = ±16.

Led to: → section 3, threshold law; → LOG 032

What I learned: As the number grows, the base moves up a step; I called this the universe ladder. My sentence in the compilation was: “At n=11 it starts again in the universe inside it… perhaps the goal is to reach that universe’s 11.”

LOG 025 · 2-3 September 2026 · C/T/A signature

Intuition: I imagined a template that marks six-digit octal numbers as even, odd and prime through Horner expansion. I was looking at 15 examples that began with C and ended with A.

Research and test: I read six-digit octal numbers step by step with Horner expansion; I marked each intermediate value C (even), T (odd) or A (prime), using the Turkish initials.

Outcome: FAILS. All 242 out of 242 signatures contain both primes and composites; 500021_8 and 500023_8 have the same signature. One “A” in my handwritten table was also wrong: $400037_8 = 131103 = 3 \times 43701$.

Led to: → today’s staged shadow and CLIMB (section 3)

What I learned: This is where I learned Horner reading.

LOG 026 · 2-3 September 2026 · Theory compilation

Intuition: I noticed the connection between the number universe I had built and the octal representations of the Mersenne forms for exponents from 1 to 12. One less than 23 was 22 = 11x2, and one less than 89 was 88 = 11x8. I read both as numbers leaving remainder 1 when divided by 11.

Research and test: I gathered C/T/A, three leading digits, the factor law, the screw and the box in one compilation.

Outcome: KNOWN IN THE LITERATURE. What I saw in 23 and 89 was Fermat and Euler's factor law (T03_factor_law).

Led to: → LOG 029

LOG 027 · 2-3 September 2026 · The square at the end of Lucas-Lehmer

Intuition: I thought of numbers as objects trying to become cubes, decreasing by 1 when they could not find a piece. I had the intuition that a square had to form in the end.

Outcome: The existence of a general modular square does not distinguish primality. This is not the same criterion as the final step of the Lucas–Lehmer sequence; the undefined "square seal" remains UNTESTED in T10. The square exists for every odd p: $(2^{((p+1)/2)})^2 = 2^{(p+1)} \equiv 2 \pmod{M_p}$. Even for composite M_{11}, $64^2 \bmod 2047 = 2$ (T10_square_seal).

What I learned: An equality being true everywhere does not make it a discriminator.

LOG 028 · Night of 4 September 2026 · Leading digit and colour gate

Intuition: In octal representations of Mersenne primes I saw 1 or 3 at the beginning, followed by 7s. I wanted to build a gate from colours using this notation of R or Y followed by White. In that note I was referring to two numbers I was looking at through 3 and 3 copies of 7. In the same examples I was also following how 23 and 89 were one more than a multiple of 11: k*11+1. I thought of 1, 3, 7, 17, 37, 77 as a representation for Mersenne numbers; 3777 corresponded to p = 11. I connected this to Yellow appearing when G is added to R, and White when blue is added too. I wanted to carry 63! in hardware as a package in CT-CPU assembly, without expanding it.

Outcome: HOLDS. The leading digit depends on p modulo 3: 1 if p = 3t + 1, and 3 if p = 3t + 2 (T05_137_octal_head). I did not discover the reason; I learned it later.

Led to: → LOG 031 (package without expanding)

What I learned: The leading digit does not distinguish primality: M_5 is prime, M_{11} is composite, and both start with 3 (T06_head_digit_primality, fails).

LOG 029 · 5 September 2026, 00:48 · p = 37 and the first large round

Question: I wanted to see how p = 37 was written in the 1-3-7 representation.

Research and test: I tried 323 claims by two separate routes.

Outcome: OPEN. 302 passed, 3 failed, 18 were undecided. The quadratic generator $M^2 + 10M + k$ failed: 42 for M = 3, k = 3 (T14_quadratic_generator).

Led to: → LOG 080 (the three remaining claims)

LOG 030 · 7 September 2026, 03:55 · The signature filter broke

Question: Instead of examples that worked for individual numbers, I was looking for an algorithm or method covering all of them.

Research and test: I had built a fixed set of 44 signatures from the first 50 Mersenne exponents.

Outcome: FAILS. The 51st exponent (82,589,933) and the 52nd (136,279,841) fell outside the set; 116 of the 649 composite controls passed through the filter. The signature was actually the residue class of p modulo a particular number (T17_signature_filter).

Led to: → LOG 031

LOG 031 · 7 September 2026, 04:12 · Packed sieve

Question: I questioned whether the sieve I had built for Mersenne primes worked. At 04:37 I also wondered whether I could accelerate the Lucas–Lehmer steps by writing only their progression.

Research and test: My decision: keep M_P as a package without expanding it. Building on the two files I had written at 04:07: M_P = (h + 1) · 8^t − 1, t = ⌊(P − 1)/3⌋, h = 2^s − 1. Candidate divisors q = 2kP + 1 and q ≡ ±1 (mod 8); the remainder of M_P modulo q comes from the package. At 04:22 I had the sieve written as plain text. The candidate search belongs to the same family as GIMPS's search for small factors.

Outcome: FAILS as a verdict. The sieve is a filter: the divisor 13367 of M_41 (k = 163) is absent from the first 64 candidates (T11_packed_sieve). The package notation itself held (T05_137_octal_head).

Led to: → LOG 032, → section 3, package notation

LOG 032 · 7 September 2026, 05:18 · The 11 scale

Research and test: I added a scale model that reads the package in base 2^11. I found no source for the combination of the package and the 11 scale.

Outcome: OPEN.

LOG 033 · 7 September 2026 · A zero-free base

Intuition: I wanted to write the number in a base with no 0 in its representation. I was thinking of everything through the notation 11: in unary, 1 and 1, that is, 11; for that reason, I was reading 2 as the first pair.

Outcome: FAILS. P = 23 and P = 67 allow zero-free representations, but M_P is composite.

Led to: → LOG 040

LOG 034 · 7 September 2026 · Carry count

Research and test: I counted how many carries arose during addition; I called this the carry count.

Outcome: KNOWN IN THE LITERATURE. Kummer's 1852 theorem: the number of carries in addition in a prime base gives that prime's exponent in the binomial coefficient.

LOG 035 · 8 September 2026, 18:22 · p = 35 in octal

Intuition: I did not see the leading 7 as an exception. I was connecting the notation 1, 11, 111 without 0 to the appearance of 1 and 3 after 7, 77. I thought of 67 as the saturation of 11 and attributed the jump 6x11+1 to that saturation.

Question: I wondered whether this representation would offer an advantage when Hunter worked in octal.

Led to: → LOG 037

LOG 036 · 10 September 2026 · Fitting something in between

Intuition: I was thinking about inserting two 0s into 11 to obtain 1001, about the square of 101, and about representations such as 11001. I was looking for a special form of addition that opens a gap and puts something into it.

Question: I considered writing 13 as 1 11 or 01 11. I asked whether, instead of writing 17 as 10001, I could package it as something like 1 111 and leave a reader to interpret it.

Research and test: I wrote the number as [D1, D0, B], with a base tag.

Outcome: FAILS. The best code takes 11 bits, versus 8 bits for plain binary; it saves no bits.

Led to: → LOG 039

LOG 037 · 12 September 2026 · p and 11

Intuition: I thought q might be composite when p was related to 11, and certainly prime when that relationship was absent.

Outcome: OPEN. I could not explain why; I did not test the “guaranteed” part.

LOG 038 · 15 September 2026, 08:02 · Reversed Mersenne P

Research and test: A note I received contained a pattern: for P = 20996011, reverse(P) = 11069902; √356798503 = 18889.11069902… and 18889 = 13 × 1453. The note was conversation output; the record of how I reached the pattern is not on this computer. I wrote my formulas and what I wanted to an AI model, because my mind kept jumping from branch to branch and I had no time to calculate. I am not Ramanujan. I had left the note and asked that it be investigated only in terms of Mersenne and Hunter.

Outcome: FAILS. The arithmetic is correct but is not specific to Mersenne: the same rule also found prime q for non-Mersenne 23, 29, 101, 1000 and 4567. The claim “every reversed P gives a special prime structure” broke for P = 2, 3, 5, 7. reverse(p) does not generate primes: 19 → 91 = 7 × 13, 61 → 16, 127 → 721 = 7 × 103. Round: of 40 claims, 27 passed, 9 failed, 4 were undecided (T15_reverse_p_general).

Led to: → LOG 039, → LOG 040, → LOG 085

What I learned: Statistics are not a basis for a verdict; I will work backwards from what is known with certainty.

LOG 039 · 15 September 2026 · The [D]101 family

Intuition: I wanted to work without propagating carries immediately. I imagined this as another number universe, similar to unary, made of sequences of 10s.

Research and test: The note contained [11]101: 11 in the upper digit of a binary representation, with value 11 · 8 + 5 = 93 = 3 × 31. It turned out to be the p = 5 member of a family: for D = 3 · 2^(p−3) − 1 + k · M_p, [D]101 = (3 + 8k) · M_p. The underlying idea—putting a value larger than the base in a digit and delaying the carry—exists in Avizienis’s 1961 redundant number systems and in carry-save adders. I have not seen the family itself elsewhere.

Outcome: HOLDS. The proof was written on 10 October (T01_d101_family).

Led to: → section 3, the [D]101 family

LOG 040 · 15 September 2026, 14:19 · Base^0

Intuition: I objected to making exceptions for the initial 2 or the first 2 digits. I sensed that this might relate to Base^0, something I thought I was overlooking in positional notation. In unary I read 2 as 1 and 1, that is, 11, and thought of it as the first twin.

Question: I asked myself what primality means in unary, in base 1.

Outcome: KNOWN IN THE LITERATURE. The testable part is a last-digit rule: if the base is divisible by a prime, the last digit tells whether the number is divisible by that prime. This is the rule we learn at school for 2 and 5 in decimal.

Led to: → LOG 046, → LOG 084

LOG 041 · 15 September 2026 · Fibonacci

Intuition: First I thought the Fibonacci formula might be wrong. Then I tried to see a pattern in which the increases themselves increase, rather than simply adding the previous number. I described this through an analogy with acceleration, then acceleration of acceleration, as increases carrying into increases.

Outcome: FAILS. The verdict “wrong formula” fell away. The increase carrying into the increase turned out to be right: every difference of Fibonacci is Fibonacci again.

LOG 042 · 16 September 2026 · Scott cpu

Research and test: I wrote “Scott cpu” in my note. J. Clark Scott’s book “But How Do It Know” is the only source that prompted an idea beforehand.

Led to: → LOG 044

LOG 043 · 19 September 2026 · Ulam band

Research and test: I wrote the diagonal as three fixed digits read in an increasing base; I derived two shift rules. The Ulam spiral has been in the literature since 1963; the record does not show how I arrived at it.

Outcome: FAILS. One broke at k = 5: the rule $4k^2 + 10k + 5 - 10\lfloor k/2 \rfloor$ gave 155 − 20 = 135 = 5 × 27, the other at 299 (13 × 23). The rule “one of n, n − 10, n − 20 is prime” was true for all 36 out of 36 numbers tried; it therefore distinguished nothing.

LOG 044 · 25 September 2026 · HunterCPU

Research and test: I wanted to see a simple Hunter like the one in Scott’s book. I built a simple Hunter model from NAND gates.

Outcome: HOLDS. It computes 6 × 7 = 42 in 268 steps. The 268 ticks are the model’s clock steps; this is not a speed measure, but shows that the program finishes correctly.

Led to: → the pure CPU book (1 October)

LOG 045 · 1 October 2026, 00:57 · Binary folding

Intuition: I was comparing 11, 1011 and the representation 1001 of 9 through division and folding; I thought 1001 needed to be folded twice. I thought I saw a link to Mersenne primality in the representation 1101 of 13; the same trace did not appear for 17. I tried to fold the representation 11111 of 31 onto itself until it reduced to a single 1. In the representation 1111111 of 127, I looked at divisions such as 111 1 111, 101 1 101 and 101 101 101, connecting them to 3 times 121. I was looking for a pattern related to primality in 101, 111 and 0 01 11 111 101.

Question: I asked whether the representation 11111 1 11111 was Mersenne.

Research and test: I wrote the p values in binary and folded them through the middle. If a representation reduced to a single 1, I called it “folding evenly.”

Outcome: OPEN. It agreed with the primality of M_p at 13, 31 and 127. It did not agree at 17: I had said “it does not work,” whereas M_17 = 131071 is prime. Why does it hold for some p values but not for 17? I could not explain it.

Led to: → LOG 046

LOG 046 · 1 October 2026 · Objection to “p >= 2”

Intuition: I objected to the condition p >= 2 in the Mersenne rules. I was trying to connect the evenness of 2 to reading it as 1 and 1 within a particular scope.

Outcome: OPEN.

LOG 047 · 10 October 2026 · 182 theories

Research and test: I gathered everything into a tested list of 182 theories. I wrote the proof of the [D]101 family.

Outcome: HOLDS. In a 40-card test, the stated verdict was confirmed on 39 cards; one has no test.

Led to: → this document: section 3 (what held) and section 4 (tests)

LOG 062 · 11 October 2026 · The ladder in exponential form

Intuition: I saw 121 as a zoomed-in version of 11. Following the same thought, I read 1331 in base 4 and 14641 in base 7.

Question: I wondered how far I could continue the sequence 1, 3, 4^2, 5^3, 8^4, 12^5, 22^6 by choosing the base to be 1 more than the largest coefficient each time.

Research and test: I wrote each row as a power of the atom. The atom is 2 more than the central number, the base 1 more; in row n, the number is atom^n. From 0 to 40 every row was written cleanly in its own base. In the base below it, every one broke.

Outcome: HOLDS. The sequence of central values exists in the literature (OEIS A001405).

Led to: → LOG 063

What I learned: I saw that 22^6 is written as 1 6 15 20 15 6 1 in base 21; I could read these coefficients directly.

LOG 063 · Night of 12 October 2026 · The +n rule broke; Catalan appeared

Question: After 5^3 I used 5+3=8 to reach 8^4, then 8+4=12 to reach 12^5. But where I expected 12+5=17, I got 22^6; I was looking for the rule behind this change.

Intuition: I thought a unary step or phase difference seemed to enter somewhere.

Research and test: I placed the increase at each step beside the numbers in the row. “+n” held only at n = 0, 1, 3, 4, because n is the second number in the row. The actual increase is the middle number or its neighbour. I could gather everything into one formula:

```
a_(n+1) = 2·a_n − 2 − (1 − d_n)·Cat_(n/2)
```

The missing piece turned out to be a Catalan number: middle minus neighbour. I checked whether it resembled Lucas–Lehmer; it does not—that is quadratic, this is linear.

Outcome: HOLDS; this notation is mine. The underlying binomial relation is KNOWN IN THE LITERATURE.

Led to: → LOG 065; later I wrote it without Catalan (section 3, Ladder step)

LOG 064 · 12 October 2026, 03:30-03:55 · Two triangles, 3^−1

Intuition: I looked for a relationship between 3^0 at n=0 and 3^-1 = 1/3, which I imagined at the preceding step. I pictured a second Pascal triangle extending upwards as 010, 0110, 01210. I thought of −1 embedding itself and emerging again through the analogy of a lining.

Research and test: I extended Pascal to negative rows and drew it from −15 to +15. Row −1 extends infinitely as 1 −1 1 −1; the absolute magnitude of its coefficients is bounded by 1. Here “bounded” does not mean the row has finite length. From there I arrived at the interpretation base 2, atom 3, that is, 3^−1 = 1/3. I cannot use the formal power series of the negative row as a numerical sum at x = 2: the series $1 − x + x^2 − …$ converges only for $|x| < 1$. At x = 2, the value 1/(1+x) = 1/3 is the evaluation of the rational expression, not the sum of this infinite series. Adding Pascal and minus Pascal leaves zero. The transition from 00 to 010 does not follow from the rule; 1 is like a seed emerging from zero.

Outcome: HOLDS. Negative rows themselves are KNOWN IN THE LITERATURE (extended Pascal).

Led to: → the drawing Pascigon.pdf

LOG 065 · 12 October 2026, 04:01-04:23 · Difference formula and flip-flop

Question: I wanted to examine the difference b.n(x) - b.n(x-1) by subtracting the a or b number of the preceding level from b.

Intuition: Instead of calling the states even and odd, I wanted to keep the state as 1 and 0. I thought of this as a counter working like a flip-flop.

Research and test: I calculated the differences from 1 to 30. Instead of even and odd, I used a counter alternating between 0 and 1. This was the recurrence formula I had previously been unable to find:

```
d_0 = 0,  d_(n+1) = 1 − d_n
mid_n = C(n, (n − d_n)/2)
b_n − b_(n−1) = C(n − 1, (n − 2 − d_n)/2)
```

It held for all 30 out of 30 rows. I did not call the middle “M”; I called it “mid”, because M belongs to Mersenne.

Outcome: HOLDS.

Led to: → LOG 066

LOG 066 · 12 October 2026, 06:00 · Reading 254^10

Intuition: Subtracting 2 from 254, I saw that 252 was the central coefficient. I thought 1 would appear 5 digits from the middle and 10 would appear 4 digits away. I had reached the formula n·(n−1)/2 for the third digit; I had not yet found the fourth. I was also considering moving forward by addition, using help from the previous coefficient.

Research and test: I went step by step from the middle to the edge; each number is (n − k)/(k + 1) times the previous one. I also found the 4th digit I had been missing: n(n−1)(n−2)/6 = 120. 254^10 is written in base 253 as 1 10 45 120 210 252 210 120 45 10 1. I also went from row 0 to 30 using only addition: computing half and mirroring required 225 additions and no multiplication.

Outcome: HOLDS. The rule itself is KNOWN IN THE LITERATURE (binomial); I arrived at it by reading the row from the middle.

Led to: → LOG 067

LOG 067 · 12 October 2026, 06:09-06:20 · Split it; the base does not matter to me

Intuition: I was sure that the coefficients were 1 6 15 20 15 6 1. I wanted to keep the coefficient sequence separate from the base and choose the base value later.

Research and test: I split 336^25 in base 335 as 10 + 10 + 5. I multiplied the rows and propagated the carry at the end. All 26 digits matched the direct calculation. The value rule holds in every base: Σ C(n,k)·b^(n−k) = (b+1)^n. I keep the number as the pair (b, n); multiplication becomes addition of exponents.

Outcome: HOLDS. The underlying pieces are KNOWN IN THE LITERATURE: Kronecker substitution, carry-save, LNS/Zech (turning addition into multiplication).

Led to: → LOG 068

LOG 068 · 12 October 2026, 06:27-06:33 · A given number

Question: I investigated how to relate a given number such as 26321361382 to a Pascal form in some base.

Intuition: I thought I could treat the given number as mid.

Research and test: I made the number a bound on the middle: base N + 1, atom N + 2. As long as the base stays larger than the row's largest central number, the row is written cleanly. It also determines which row to use: the largest row whose middle does not exceed N. For this number, that is row 37:

```
1 37 666 7770 66045 435897 2324784 10295472 38608020 124403620
348330136 854992152 1852482996 3562467300 6107086800 9364199760
12875774670 15905368710 [17672631900] [17672631900] ... mirror
```

So (N + 2)^37 in base N + 1 is exactly this. There is no multiplication: the ends are 1, the row is mirrored, and the values between come from addition. The middle of row 38 is 35345263800, exceeding N; that is where the digits carry (T20_middle_by_addition).

Outcome: HOLDS up to row 37. Generalising the base from 11 to every number is KNOWN IN THE LITERATURE (Jiu and Vignat 2016; Islam and colleagues 2020).

Led to: → LOG 069

LOG 069 · 12 October 2026, 06:34 · Does this exist elsewhere? (2)

Question: Has this route been found before, and how niche is it?

Research and test: I searched only with general terms, without writing my own formula. Islam and colleagues had chosen the base from the central coefficient in 2020 (arXiv 2005.14490), but as a power of 10. The smallest base was recorded in A051920, the atom-power ladder in A193242, and the step rule in A001405.

Outcome: KNOWN IN THE LITERATURE. T08 is also partly in the literature. What remains for me is the d counter and the (b, n) data layout; in other words, implementing it in a processor.

What I learned: I am also trying to build a processor. For me, the difference is in the architecture, not the mathematics.

LOG 070 · 12 October 2026, 06:46-06:51 · Swap and Mersenne bases

Intuition: I split the exponent in 2^7 as 7 = 3x2+1 and looked at the notation 2^(3x2+1).

Research and test: I read 128^9 = 2^63 by swapping the exponents. `(2^7)^9`, `(2^9)^7` and `(2^21)^3` all give clean rows. The bases are 2^k − 1, that is, Mersenne numbers: 127 is prime, 511 and 2097151 are not. Up to 2000 there are only three rows whose atom is a power of 2: n = 2, 4, 9 (exponents 2, 3, 7). The chain 2 → 3 → 7 resembles the beginning of the Catalan–Mersenne sequence, but does not continue with 15 and 127. 7 + 2 = 9 is also a coincidence.

Outcome: HOLDS. Whether the chain is finite remains OPEN.

Led to: → LOG 072

LOG 071 · 12 October 2026 · A chain in base 3

Question: What remains when powers below a power are subtracted as a chain?

Research and test: I took b^n at the top, subtracted the chain from b^(n−1) down to b^m, and finally subtracted c copies of 1:

```
value = b^n − (b^(n−1) + … + b^m) − c = b^n − (b^n − b^m)/(b − 1) − c
b = 2 :  value = 2^m − c             the top disappears completely
b = 3 :  value = (3^n + 3^m)/2 − c   the top is halved
```

With m = 1 in base 3:

```
                        c = 0    c = 1
3² − 3                      6        5
3³ − 3² − 3                15       14
```

```
3⁴ − 3³ − 3² − 3            42       41
3⁵ − 3⁴ − … − 3            123      122
3⁶ − 3⁵ − … − 3            366      365
3⁷ − 3⁶ − … − 3           1095     1094
3⁸ − 3⁷ − … − 3           3282     3281
3⁹ − 3⁸ − … − 3           9843     9842
3^10 − 3⁹ − … − 3        29526    29525
```

The c = 0 column is (3^n + 3)/2, and the c = 1 column is (3^n + 1)/2. Subtracting the final 1 is the same as extending the chain down to 3^0; this gives the simple sequence a(n) = (3^n + 1)/2 → 1, 2, 5, 14, 41, 122, 365, 1094. In base 3 it is written as 11…12. One step gives the next term: a(n+1) = 3·a(n) − 1. Since 3^5 = 243 = 2·121 + 1, the remainders of a(n) modulo 121 cycle through 1, 2, 5, 14, 41 every five terms; a(5) = 122 = 121 + 1 returns to the start. Therefore a(n) is never divisible by 11. The reversal between 14 and 41 is a coincidence; it occurs only there up to 300.

Outcome: HOLDS (T22_base3_chain). The sequence is KNOWN IN THE LITERATURE (OEIS A007051). The fact that 3^10 leaves remainder 1 modulo 121—that 11 is a base-3 Wieferich prime—is also KNOWN IN THE LITERATURE; the other one is 1006003, and I ran both.

Led to: → links back to 121 in LOG 020

LOG 072 · 13 October 2026 · A power of 2 in a Mersenne base

Research and test: I generalised 128^9 from LOG 070. In base M_p = 2^p − 1, 2^p is written “11”; therefore 2^(p(p+2)) in this base is row p+2 of Pascal’s triangle. I generated the digits from one another and reconstructed the value with Horner:

```
int n = p + 2;                      /* row number            */
uint64_t b = (1ULL << p) - 1;       /* base = M_p            */
uint64_t R = 1, v = 0;              /* leftmost 1, value     */
for (int k = 0; k <= n; k++) {
    v = v * b + R;                  /* v = v*base + digit    */
    R = R * (n - k) / (k + 1);      /* next digit            */
}
/* p = 7:  1 9 36 84 126 126 84 36 9 1,  v = 2^63 */
```

This uint64_t example is given only for p = 7; for larger exponents the full Horner value does not fit in 64 bits. General computation requires multiprecision integers.

At p = 5 the middle exceeds the base and the row carries; for p ≥ 7 it is always clean. From p = 7 to 37, p+2 is the largest row whose middle remains below the base; from 38 onward p+3 fits as well. I tried the first 30 Mersenne exponents. At the thirtieth, p = 132049, all 132052 digits were smaller than the base, and the value matched 2^(p(p+2)) modulo three primes (2.4 seconds with GMP). Primality plays no role: at p = 8 the base 255 is not prime, yet the row is still clean.

Outcome: HOLDS (T21_mersenne_row). The underlying binomial theorem and the generalisation of powers of 11 to every base are KNOWN IN THE LITERATURE (Jiu and Vignat 2016). It does not distinguish Mersenne primes from other Mersenne numbers.

Led to: → LOG 073

The proof recorded here covers odd p ≥ 7. This package checks even exponents through T21 over p = 7..61. The extended proof for every integer p ≥ 7 is in Pascal Mersenne Rows.

LOG 073 · 14 October 2026 · Powers of 2 in a Mersenne base: a clean Pascal row

Question: Does this row stay clean for large Mersenne primes too, and why?

Research and test: I proved three things. Let n = p + 2 and x = (p + 1)/2.

```
T1 (value)    Σ C(n,k)·M_p^(n−k) = (M_p + 1)^n = 2^(p(p+2))
T2 (clean)    for odd p ≥ 7, middle < M_p; no digit carries
T3 (middle)   middle = C(2x+1, x) = (p + 2)·Cat_x
```

T1 is the binomial theorem. I showed T2 by induction: at n = 9, 126 < 127. Moving forward two rows multiplies the middle by 2(2x+3)/(x+2), which is smaller than 4; the base grows by more than a factor of 4 (M_(p+2) = 4·M_p + 3). Thus the middle can never catch the base. It fails for p = 3 and 5 (10 > 7, 35 > 31). In T3, the row's 2nd digit divides the middle: at p = 7, 126 = 9·14; at p = 13, 6435 = 15·429.

I can find the middle's factors without touching the large number, using only small divisions (Legendre). Every prime greater than (p+3)/2 and at most n occurs once in the middle. If (p+3)/2 itself is prime, it is absent: at p = 7, 5 does not occur in 126.

I computed this for all 52 Mersenne exponents in my 13–14 October working list. That is the scope of that experiment list, not a claim about the current record list. The large table calculations are not repeated by this package's journal_tests.c. At the largest, p = 136279841, the middle digit is 136 million bits and 11 bits below the base. I tabulated the decimal and octal representations of the middle digits, their factors, and the exponents of the first seven primes; all also agreed with a second calculation.

Outcome: HOLDS; the proof above applies to every odd p ≥ 7 (T21_mersenne_row also tests even and odd exponents over p = 7..61). The tools I use are KNOWN IN THE LITERATURE: the binomial theorem, Legendre's formula, C(2x+1, x) = (2x+1)·Cat_x. What I did was choose base M_p and row p + 2, and show that it has no carries for every odd p ≥ 7. It does not distinguish primality: M_11 = 23·89 gives the same result.

Led to: → LOG 074

What I learned: No calculation is needed to write a huge power of 2 in a Mersenne base; its digits are known in advance.

LOG 074 · 14 October 2026 · The exponents' own representations

Question: What do the Mersenne exponents themselves look like in binary?

Research and test: I wrote the 52 exponents in binary, octal and signed binary (NAF), placing them beneath one another in a 28-column grid. Each block of 1s in binary is a Mersenne difference: 2^(a−1) + … + 2^b = M_a − M_b = 2^b·M_(a−b). Each exponent thus becomes a sum of Mersenne numbers; for example, 521 = (M_10 − M_9) + (M_4 − M_3) + M_1. Only four exponents have a single block—that is, are themselves Mersenne: 3 = M_2, 7 = M_3, 31 = M_5, 127 = M_7. I also expanded the differences between consecutive exponents in the same way.

Outcome: HOLDS as a representation. The four single-block exponents correspond to double Mersenne primes (M_M_2, M_M_3, M_M_5, M_M_7), KNOWN IN THE LITERATURE. The chain 2 → 3 → 7 → 127 is the Catalan–Mersenne sequence; the 2 → 3 → 7 in LOG 070 connects here. I found no rule in the pattern that predicts the next exponent.

Map of the chain

I also drew the family tree. A solid arrow (->) marks a link supported by a written record. A dotted arrow (..>) marks a possible link: the content matches, but I could not find a written sentence connecting the two.

```
[ROOT 1] Computing with colour and light (21 Apr draft, 8 Jun final)
  +-> xbit: 3 bits = 1 octal digit = 1 colour (8 Jun)
       +-> octal base (8 Jun; 31 Aug "base 8 approved")
       +-> eight type tags (13 May) -> TYPE cell
       +-> Mersenne xbit pattern W/RW/YW (16-19 Jun)
            +-> octal Mersenne patterns, M_p = leading digit + 7s (26 Jul)
                 +-> {1,3,7} language (2 Aug)
                      +-> 1-3-7 tally universe (5 Aug)
                           +-> three leading digits (31 Aug)
                           +-> package M_P = (h+1)·8^t-1 -> PACKED SIEVE (7 Sep)
```

```
                            +-> [D]101 family (15 Sep; proof 10 Oct)

[ROOT 2] R/Y: Mersenne exponent mod 3 (21 Apr) ..> Mersenne xbit pattern (16-19 Jun)

[ROOT 3] LOG mode: multiplication = index addition (23 Apr) ..> SHADOW

[ROOT 4] SHADOW: multiplication = exponent addition (9 Jun)
  +-> resolver (20 Jun) -> digit flow (22 Jun) -> folding channels (4 Sep)
  +-> lazy shadow (1-2 Jul), RR was the register holding the rough number (until early August); it
became a separate field on 2 August
  +-> "shadow does not help LL" (31 Aug) -> p = 3q + r ..> package (h, t)
  +-> carry-free universe (1 Sep) ..> overflowing digit, [D]101 (15 Sep)

[ROOT 5] Mersenne and Lucas-Lehmer (separate repositories, June-August)
  +-> counting machine, FOLD (between 26 Jul and 2 Aug)
  |    +-> bit folding wins (31 Aug) -> screw and turns (2-3 Sep)
  +-> factor law q = 2kp+1 (26 Jul; my words 2-4 Sep) -> PACKED SIEVE
  +-> 2p+1 gate (29 Jul) -> PACKED SIEVE
  +-> signature filter broke (7 Sep) -> PACKED SIEVE

[ROOT 6] 121 and Pascal
  +-> 121 = 11 times 11 (31 Aug) -> 1331 in every base (1 Sep)
  |    -> 11 IS A FORMULA (2 Sep)
  |         +-> ladder, threshold C(n,n/2)+1 (2 Sep)
  |         |    -> ladder step, flip-flop (11-12 Oct)
  |         |    -> power of 2 in a Mersenne base (13 Oct) -> clean row, 52 exponents (14 Oct)
  |         +-> unary, "the most important number is 3" (1 Sep) -> Base^0 (15 Sep)
  |         +-> 11 scale (7 Sep) -> PACKED SIEVE
  +-> Horner chain "235" (30 Aug) -> C/T/A signature (2-3 Sep, failed)
  +-> base-3 chain, period five at 121 (12 Oct)
  +-> reading in a neighbouring base (1-10 Sep) -> REVERSED MERSENNE P (15 Sep) -> [D]101

[ROOT 7] J. Clark Scott, "But How Do It Know" (16 Sep)
  -> HunterCPU (25 Sep) -> pure CPU book (1 Oct)
```

Looking at the tree, three nodes generated the most ideas: 121 and Pascal, shadow, and the 1-3-7 universe. The packed sieve is where the most branches meet. 1-3-7, the factor law, the 2p + 1 gate, folding, the defeat of the signature filter, and the 11 scale all flow into it.

---

3. What held: the formulas I arrived at clearly

This section contains only what held. What failed and what was already known are marked FAILS and KNOWN IN THE LITERATURE in the journal, and are listed separately in section 4. By “clearly” I mean that it passed the test and I can show why it is true with a short calculation. I have also given each result’s test name. The same names appear in journal_tests.c.

The formulas begin with the note in LOG 022. When the base increases by one, the number does too, and the representation stays the same: in base b, (b + 1)^3 = 1331. This is true starting at b = 4; in base 3, 3 cannot be a digit, so 1331 cannot be written. The threshold law below generalises that note.

Ladder step

For a long time I could not find how the ladder goes from one step to the next. Eventually it came down to a single addition:

a_(n+1) = a_n + kom_n, a_0 = 3

Here n is the row number in Pascal’s triangle. a_n is that row’s atom: 2 more than the largest number in the middle. kom_n is the neighbour to the left of the middle; if there are two equal middle numbers, the neighbour equals the middle itself. A counter keeps track of whether the row has one or two central entries. It alternates 0, 1, 0, 1, like a flip-flop.

The general form without Catalan, all with one counter (I take C(n,k) outside the row to be 0; for example, C(0,−1) = 0):

```
d_0 = 0,   d_(n+1) = 1 − d_n          (counter: 0, 1, 0, 1, ...)
mid_n = C(n, (n − d_n)/2)             (middle)
kom_n = C(n, (n − 2 + d_n)/2)         (neighbour)
a_n   = mid_n + 2                     (atom)
a_(n+1) = a_n + kom_n,   a_0 = 3
```

It says the same thing as the Catalan form in LOG 063, but with addition instead of subtraction. The reason is Pascal's rule: the middle of the next row is the sum of this row's middle and its neighbour.

Row 6 is 1 6 15 [20] 15 6 1. The middle is 20, the atom 22, the neighbour 15; the next atom is 22 + 15 = 37. Row 7 has two 35s, so the neighbour is 35 itself; the atom is 37 and the next is 37 + 35 = 72. I continued this way up to row 1000, and it always held (archive record; T18 in this package checks only n ≤ 60). No multiplication is needed, only addition.

Then I investigated. The step rule was already written exactly this way in OEIS entry A001405 (Hanusa 2003). The smallest base is in A051920 (Sloane 1999), and the atom-power ladder in A193242 (Povolotsky 2013). So the mathematics is in the literature. What I want is to implement this step with an adder and a flip-flop; I have not yet built that circuit. The neighbouring coefficient must also be generated or stored; counting the single addition does not explain the memory and computation costs of the whole circuit.

Threshold law and ladder

threshold(n) = C(n, ⌊n/2⌋) + 1

I was looking at 11^n, that is, (b + 1)^n. If this number can be written in base b without carries, its representation is exactly row n of Pascal's triangle. The base must be at least threshold(n), and that is sufficient too. At n = 4 the row is 1 4 6 4 1; the largest coefficient is 6 and the threshold is 7. Decimal base is greater than 7, so 11^4 = 14641, the row itself. At n = 5 the row is 1 5 10 10 5 1; the central 10 cannot be a single decimal digit, and the threshold is 11. In decimal, 11^5 = 161051, so the row is disrupted. In base 11, however, 12^5 is again written 1 5 10 10 5 1, with each coefficient a single digit. This is where the ladder comes from. From n = 1 to 6, the thresholds are 2, 3, 4, 7, 11, 21. As n grows, the number no longer fits its own universe and must move to a higher base.

Why is it true? The binomial expansion is (b + 1)^n = Σ C(n, k) · b^k. If all coefficients are smaller than b, each is a valid digit. Since a number's base-b representation is unique, the representation is exactly the row. The largest coefficient in the row is the central C(n, ⌊n/2⌋). If b is larger, the representation is the row. If b is equal to or smaller than it, the middle coefficient cannot be a digit; a carry arises and the representation departs from the row. The sequence of central coefficients is known (OEIS A001405); I read the threshold by this route.

Its test name is T08_threshold_law. For n = 1..15, 44 base checks at the threshold, one above and one below; it held.

The [D]101 family

[D]101 = (3 + 8k) · M_p, where D = 3 · 2^(p−3) − 1 + k · M_p; p ≥ 3 and k ≥ 0 are integers.

The notation [D]101 means putting D above binary 101, that is, 5; its value is 8D + 5. With D chosen this way, the resulting number is always a multiple of M_p. For p = 5, k = 0, D = 3 · 4 − 1 = 11 and [11]101 = 11 · 8 + 5 = 93 = 3 × 31.

Why is it true? 8D + 5 = 3 · 2^p − 8 + 8k · M_p + 5 = 3 · (2^p − 1) + 8k · M_p = (3 + 8k) · M_p. The converse holds too: if 8D + 5 = t · M_p, then t ≡ 3 (mod 8) is necessary. Since M_p ≡ −1 (mod 8) and 8D + 5 ≡ 5 (mod 8), we have −t ≡ 5, hence t ≡ 3. Every multiple of this form belongs to the family. The family says nothing about primality; I supply the factor myself.

Its test name is T01_d101_family. For prime p = 3..31 and k = 0..3, 40 equalities; it held.

The 1-3-7 leading digit and package notation

M_P = (h + 1) · 8^t − 1, where P = 3t + s, s ∈ {1, 2, 3} and h = 2^s − 1

In octal, M_P is a leading digit h followed by t copies of 7. h is 1, 3 or 7 according to P modulo 3. For P = 11, 11 = 3 · 3 + 2, so t = 3, s = 2, h = 3; M_11 = 3777_8 = 2047. This is what I meant by “3777, for example, corresponds to 11.” The packed sieve keeps M_P in this notation without expanding it.

Why is it true? 2^P = 2^s · 8^t, so M_P = 2^s · 8^t − 1 = (2^s − 1) · 8^t + (8^t − 1). In octal, 8^t − 1 is t copies of 7; 2^s − 1 is a single digit, 1, 3 or 7. If P is prime and greater than 3, it is not divisible by 3, so s cannot be 3; only leading digits 1 or 3 remain.

Its test name is T05_137_octal_head. For p = 3..300, 298/298; it held.

M_p ≡ −1 (mod 8)

M_p ≡ 7 (mod 8), p ≥ 3

If p is 3 or greater, the last octal digit of M_p is always 7. M_5 = 31 = 3 · 8 + 7. Why is it true? For p ≥ 3, 2^p is divisible by 8, so 2^p − 1 ≡ −1 ≡ 7 (mod 8). This is the simplest formula; the converse direction of the [D]101 family also rests on it.

Its test name is T02_mersenne_mod8. For p = 3..79, 77 exponents; it held.

The machine: Hunter as described in this compilation

This section describes the design and archive records in the October 2026 compilation. The area, speed, cycle and large test counts below are not reproduced by journal_tests.c. They are historical model/experiment records, not newly verified benchmarks or FPGA/silicon results for this package.

The machine became simpler over time. On 30 August I decided to rewrite CT-CPU, and its name became Hunter. The old arrangement—4096 threads, MIMD, coloured gates—now remains only as history. I keep Hunter’s state at that time in one machine document.

Hunter is a single-core, single-thread machine with 8 registers. A number consists of eight octal digits; the largest is 77777777_8, or 16,777,215. Each register contains eight small subregisters: positional representation (rm), four shadow rows (rs1..rs4), a carry row (rc), scale (rpow) and type (rtyp). Each has eight cells of three bits. A complete register is 192 bits.

The shadow too has changed from its original form. The first shadow directly factored the number into primes. Hunter’s shadow reduces the number stage by stage in octal: val = smooth × (8 × alt + a). Each row contains the exponents of seven small primes (2, 3, 5, 7, 11, 13, 17) and a remainder digit. This generalises Horner’s scheme. If smooth were 1 at every stage, I would have an ordinary octal number.

The instruction set has eight opcodes: LOAD, MOV, the ADD family (ADD, SUB, CMP), the MUL family (shadow multiplication and division, square root, square, shift), J (conditional jump), LD, ST and SYS. An instruction is four xbits, or 12 bits. My design goal is to connect opcodes directly to control paths without microcode. This description does not show that instruction decoding or control-generation logic is cost-free; I am not presenting a synthesised circuit here.

I report my measurements without changing them. For a 24-bit array multiplier, the conventional circuit requires 576 full adders, the shadow 30. In this model only the counted number of adders is about 19 times smaller; this is not the area ratio of the entire processor. For random pairs of 24-bit numbers, the mean full-adder cost was 529 on the conventional path and 1026 in the shadow. That is because most of a random number remains rough, and that part is multiplied conventionally (see 3.75 bits). I expect the shadow to be useful for numbers with known structure; the benefit depends on the operation, representation conversion and implementation used for comparison. The resolver that constructs the shadow is excluded from this count; in the design it typically takes 3 cycles and works without division through digit flow. The archived division comparison reported roughly threefold speed; that comparison was not repeated in this package. Converting all 2^24 numbers to shadow and back produced 0 errors. There is no silicon. Operation and gate counts are limited by the model used; hardware implementation and measurement are needed for actual clock frequency, power and total system area.

What held in the machine

These are simulation results from the archive; this publication's 22 cards do not include those machine tests.

There are also two machine measurements rather than formulas. Converting all 2^24 numbers to shadow and back to positional notation produced 0 errors (the measurement above). The simple Hunter model I built from NAND gates correctly computed 6 × 7 = 42 in 268 steps. The 268 ticks are clock steps of the model; they are not a speed measure but show that the program finishes correctly. These were measured in simulation; they have not yet been tested on a chip (to be tried with Tang Nano 9K).

Resolver: measuring while the number flows

To build the shadow, I needed to know how many factors of 2, 3, 5, 7, 11, 13, 17 were in the number. I did not want a division circuit. I streamed the number's octal digits from the left. As each digit arrived, seven channels updated their remainders: remainder ← (8 · remainder + digit) mod q. When the last digit entered, the answer was ready. In a channel where division succeeded, the quotient also streamed out and entered the next stage; each division ticked that channel's counter, and exceeding 7 carried into the next row.

When I streamed 3^{14}, I saw something: 14641, 1331, 121, 11 appeared between stages. $3^2 = 9$, written 11 in octal; the coefficients of the expansion 3^(2n) = 9^n = (8+1)^n are therefore a Pascal row. In octal, these coefficients are direct digits only while they are smaller than 8: n = 0..4 is clean, and carrying begins at n = 5 because the central coefficient is 10. For 3^14, n = 7; we cannot treat its coefficients as direct octal digits. Pascal was there again.

In the literature: remainder flow is Horner (1819), quotient flow is long division. My part is bringing them together into one flow with seven parallel channels, chained stages and octal counters that propagate carries.

The shadow's limit: 3.75 bits

When I take a random number, how large on average is the part contributed by the seven primes the shadow keeps (2, 3, 5, 7, 11, 13, 17)? For example, $744 = 2^3 \cdot 3 \cdot 31$. The part entering the shadow is $2^3 \cdot 3 = 24$; the remaining 31 is rough and goes into rc.

Each prime contributes an average of $\log_2(p) / (p - 1)$ bits to the shadow. Half of all numbers are divisible by 2, a quarter twice, an eighth three times; together these give an average of one factor of 2, or 1 bit. For 3, the mean is half a factor of 3, giving $\frac{1}{2} \cdot \log_2 3 \approx 0.79$ bits.

Prime	Mean contribution (bits)
2	1.000
3	0.792
5	0.580
7	0.468
11	0.346
13	0.308
17	0.255
Total	3.75

This sum is the mean logarithmic contribution obtained by choosing positive integers uniformly from 1..N as N grows; this prime-exponent model is not used for zero. It was measured in the archived experiment with 200,000 random numbers: the mean was 3.755 bits, agreeing with the formula. 2^3.75 ≈ 13.5 is approximately the geometric mean of the part entering the shadow, not the arithmetic mean of the number values. Most of the number remains in rough.

That is why the shadow does not fly on random data. Its strength emerges in numbers with known structure: factorials, powers, Mersenne-form numbers and calculations with known factors.

4. Tests

“I have a great deal of faith in and interest in statistics, but proceeding from things whose causes we do not know feels like alchemy to me.”

So here I list what I tried in order, from old to new. Each entry states what I tried, the file I used, the result and its counterpart in this publication. I refer to the files by their archive names. All the old files remain in place; for this publication, however, I rewrote the tests from scratch in one file. I wrote the tests in C:

```
cc -std=c99 -O2 -o journal_tests journal_tests.c -lm && ./journal_tests
```

I use four terms for the results. Holds: something I constructed by my own route was confirmed. Fails: there is a counterexample. Known in the literature: it proved correct; in my research I learned that the method had already been found and studied. Untested: a claim for which no test could be written in this publication. I did not treat statistical results as verdicts; I marked them separately.

What I tried, in order

The following list summarises the old experiment notebook. Experiments identified only by their archived filenames are not included in this package and were not rerun in this publication review. Times, energy, area, percentages and large check counts record the corresponding old run; they do not guarantee general speed on different machines or algorithms. The reproducible scope of this package is limited to the 22 cards below.

- **April.** Distribution of Mersenne exponents modulo 3 and the R/Y strip · April-July CT-CPU archive, MERSENNE/analysis/mod3_distribution.py and visualize_ry_strip.py · statistics: no bias appeared over 52 exponents; not treated as a verdict · tried.
- **April.** Single-xbit multiplication and division in LOG mode · same archive, CTCPU_CORE/RAX_ALU/log_mode_test.py · undecided: 17 multiplication and 14 division cases are written, with no run output · tried.
- **June.** First C simulation of shadow: converter and shadow multiplication · CT-CPU backup archive, Agent2Simulations/ctcpu_golge.c and ctcpu_cekirdek_test.c · held: 5 million shadow multiplications in 4.2 ms (measurement on a computer) · T13_shadow_vector (known in the literature).
- **June.** Shadow emulator, all 2^24 numbers: round trip, remainders for 7, 11 and 13, rough · CT-CPU archive, GOLGE_NIHAI_PAKET/TESTLER/test_nihai.c, test_ek.c and test_roughflag.c · held: 18/18 · tried (section 3, What held in the machine).
- **July.** Binomial multiplication with quarter-squares · CT-CPU git archive, 04_ARGE/CEYREK_KARE_DENEY.c · held: 0 errors over 3 million pairs; binomial won on small numbers · tried.
- **July.** Structural square, m = 8n + r · CT-CPU git archive, 04_ARGE/YAPISAL_KARE_DENEY.c · held: 0 errors, squaring 42-44 percent cheaper · tried.
- **July.** Octal Mersenne pattern, p = 5..127 · CT-CPU archive, 04_ARGE/MERSENNE_ARASTIRMA/GECE_ARASTIRMASI.html (calculation line inside the report) · held: 10 examples, no exception · T05_137_octal_head (holds).
- **July.** Digit flow and shadow round trip · CT-CPU archive, RESOLVER/EMULATOR/resolver_emulator.c, golge_nihai.c and TUM_TESTLERI_KOSTUR.sh · 0 errors over 117 million checks · T12_digit_flow (known in the literature: Horner).
- **July.** Adaptive precision: rendering and matrix multiplication · CT-CPU archive, 04_ARGE/ADAPTIF_TEST2/render_adaptif.c and matmul_adaptif.c · mixed: rendering 1.3 times faster, falling to

0.81 times for an ALU-heavy workload · tried.

- **August.** Combined shadow audit, 2^24 · CT-CPU archive, B_GOLGE/EMULATOR/CEM_MODELI/golge_nihai.c · held: 0 double-rough cases in 60,436,967 transitions · tried.
- **August.** Visual simulation of the Mersenne workbench · Mersenne workbench repository, CTCPU_SIMULASYON.html · found two errors (OR parser and RR resolution), both corrected · tried.
- **August.** Cube identity 8^p − 1 = M_3p and the 2p + 1 gate · Hunter archive, test sequence in 137/EVREN_137.html · known in the literature: for p ≤ 200 passing the gate, 2p + 1 divided M_p; this is Euler's law · T04_euler_2p_plus1 (known in the literature).
- **Late August.** Live shadow flow · CT-CPU git archive, 29_AGUSTOS_2026/GOLGE_AKIS_CANLI.html · held: 9/9, 0 errors in 300 random round trips · tried.
- **Early September.** First large round of 323 claims · TEST_001 experiment archive, KOD/optim/test_battery.py · 302 passed, 3 failed, 18 undecided · the following four entries are parts of this round.
- **Early September.** Quadratic generator M² + 10M + k · TEST_001 experiment archive, KOD/kume-4/A/k4_A_bolum29.py · failed: 42 for M = 3, k = 3 · T14_quadratic_generator (fails).
- **Early September.** C/T/A signature · TEST_001 experiment archive, KOD/kume-5/A/k5_A_orta1.py · failed: the signature alone does not distinguish primality · tried.
- **Early September.** Pascal threshold, n ≤ 60 · TEST_001 experiment archive, KOD/kume-3/B/k3_B_bolum13.py · correct in that range, finite trial · T08_threshold_law (holds).
- **Early September.** Screwdriver and folding count · TEST_001 experiment archive, f1_tornavida.py · failed: these are the round's remaining 3 claims · tried.
- **Early September.** Hunter Mersenne pipeline without Lucas-Lehmer · Hunter archive, MERSENNE/DEX/test_hunter_pipeline.py · 38 passed, 4 failed, 7 open; 35,936,789 checks · its parts are below.
- **Early September.** Leading digit, p = 1..1000 · same file · held: 1997 checks, 0 violations · T05_137_octal_head (holds).
- **Early September.** Factor law q = 2kP + 1 and q ≡ ±1 (mod 8) · same file · known in the literature: 313,714 checks · T03_factor_law (known in the literature).
- **Early September.** Pascal threshold, n ≤ 200 · same file · held · T08_threshold_law (holds).
- **Early September.** Fixed signature filter · same file · failed: the 51st and 52nd exponents did not enter the set · T17_signature_filter (untested in this publication: the historical signature function and fixed signature set are absent from this package).
- **Early September.** Discriminating power of the signature feature · same file · statistics: not treated as a verdict · tried.
- **Early September.** Packed sieve, second version · Hunter archive, MERSENNE/DEX/test_paketli_elek_v2.py · the composites missed by the sieve were seen not in this file but in October's theory test · T11_packed_sieve (fails).
- **Early September.** 11_b = b + 1 and unary scale · Hunter archive, MERSENNE/DEX/test_11_olcek_modeli.py · known in the literature · T09_11_base_value (known in the literature).
- **Mid-September.** Reversed Mersenne P, 40 claims · TEST_003 experiment archive, KOD/KOK/geri_muhendislik.py · 27 passed, 9 failed, 4 undecided; specificity to Mersenne failed · T15_reverse_p_general (a counterexample only to the claim that reversal always gives a prime; the entire broader historical mechanism was not tested).
- **September.** Ulam band · Hunter archive, TEORILER/ARASTIRMA_20260919/HUNTER_BANT_TABAN_DENETIM.py · counts held for k ≤ 200; the shift rule broke at k = 5 · tried.
- **Late September.** HunterCPU NAND adder · Hunter archive, SISTEM/HunterCPU/tests/cpu.test.mjs · held: 65,536 input pairs · tried.

- **Late September.** Folding and screw, 45,056 identities · Hunter archive, NIHAI_FORM_300926/06_NIHAI/KANIT/kesin_kontrol.py · known in the literature: the underlying arithmetic is multiplicative order · tried.
- **Early October.** 6 × 7 = 42 with HunterCPU · Hunter archive, HUNTERCPU_6X7_021026/kontrol.py · held: 268 steps · tried.
- **October.** 40-card theory test · Hunter theories, testler.c · the stated verdict was confirmed on 39 cards, one has no test · the square seal, 242/242 C/T/A result and missed M_41 first appeared here: T10_square_seal (undefined criterion; untested) and T11_packed_sieve (fails).
- **October.** Tests for this publication · journal_tests.c · 22 tests: 15 hold (7 known in the literature), 4 fail, 3 untested · table below.

There is no old code file for the square seal; it was first tested in October's theory test.

This publication's 22 tests

Test name	What it tests	Status	Result	Range
T01_d101_family	[D]101 = (3 + 8k) · M_p	HOLDS	40 equalities correct	prime p = 3..31, k = 0..3
T02_mersenne_mod8	M_p ≡ 7 (mod 8)	HOLDS	Correct for 77 exponents	p = 3..79
T03_factor_law	Divisors q = 2kp + 1 and q ≡ ±1 (mod 8); Fermat, Euler	HOLDS · KNOWN IN THE LITERATURE	14 prime factors met the condition	prime p = 3..31
T04_euler_2p_plus1	If p ≡ 3 (mod 4) and 2p + 1 is prime, it divides M_p; Euler 1750	HOLDS · KNOWN IN THE LITERATURE	3 eligible q values divided; p = 3 exception excluded	prime p < 100
T05_137_octal_head	In octal, M_p starts with 1, 3 or 7 and ends with 7s	HOLDS	298/298 correct	p = 3..300
T06_head_digit_primality	Does the leading digit distinguish primality?	FAILS	M_11 = 2047 is composite and starts with 3	p = 5, 11
T07_pascal_11	Powers of 11 are Pascal rows; binomial theorem	HOLDS · KNOWN IN THE LITERATURE	65 digit checks correct	n = 0..4; n = 1..30, base largest coefficient + 1 and + 2
T08_threshold_law	Threshold base C(n, ⌊n/2⌋) + 1 for carry-free representation	HOLDS	44 base checks correct	n = 1..15, threshold and one below
T09_11_base_value	11_b = b + 1	HOLDS · KNOWN IN THE LITERATURE	99 bases and 98 squares correct	b = 2..100; squares b = 3..100
T10_square_seal	Is the square at the end of LL a primality signature?	UNTESTED	Undefined criterion	small p = 3, 5, 7, 11, 13
T11_packed_sieve	Does the sieve catch all composites with k ≤ 64?	FAILS	M_41 is composite, divisor 13367 (k = 163); k ≤ 64 misses it	p = 41
T12_digit_flow	Octal Horner remainder flow gives the correct remainder; Horner	HOLDS · KNOWN IN THE LITERATURE	491,520 remainder equalities correct	n < 32768, p = 3..17
T13_shadow_vector	Tests that shadow multiplication adds exponents	HOLDS · KNOWN IN THE LITERATURE	4096 products correct	a, b = 1..64

Test name	What it tests	Status	Result	Range
T14_quadratic_generator	Does M² + 10M + k generate primes?	FAILS	42 = 2 × 21 for M = 3, k = 3	M = 3, k = 3
T15_reverse_p_general	Is the decimal reversal of a prime always prime?	FAILS	Counterexample 19→91=7×13	p = 19
T16_mersenne_infinitude	The infinitude of Mersenne primes follows from this mechanism	UNTESTED	No test written	Open problem
T17_signature_filter	A fixed signature set retains all subsequent exponents	UNTESTED	Signature function and fixed signature set absent from the package; raw exponent membership is not a signature test	Historical #51 and #52 claim
T18_ladder_step	a_(n+1)=a_n+kom_n	HOLDS	Formula and Pascal row agree	n=0..60, uint64_t
T19_base_difference	b_n-b_(n-1)=kom_(n-1)	HOLDS	Difference and neighbouring coefficient agree	n=1..60, uint64_t
T20_middle_by_addition	Given number as middle: base N+1, addition from the edge + mirror	HOLDS	0-37 clean; at 38 the middle exceeds N	N = 26321361382
T21_mersenne_row	2^(p(p+2)) is row p+2 in base 2^p − 1	HOLDS	p = 5 carries; at p = 7..37 the largest clean row is p+2	p = 7..61, 3 prime moduli
T22_base3_chain	b^n − chain − c; a(n) = (3^n+1)/2	HOLDS · KNOWN IN THE LITERATURE	6156 equalities; period five mod 121; Wieferich base 3: 11, 1006003	b = 2..10, n < 2000, q < 2·10^6

Total: 22 tests, 15 hold, 4 fail, 3 untested. I later found the sources for 7 of the 15 holding tests in the literature. The complete output is in TEST_OUTPUT.txt. The code and output are in English: HOLDS = held, FAILS = failed, UNTESTED = could not be tested, ERROR = code error. The code reports only what it computes. Literature information is in this table. T11 showed me that the sieve is a filter, not a verdict. The divisor of M_41 is 13367 = 2 · 163 · 41 + 1. If the sieve had taken k up to 163, it would have found it; it does not find it among the first 64 candidates.

---

5. The far-out ideas

Here are things I left at “could it be this way?”: ideas that failed or could not be tested. The format is the same as the journal, and the numbering continues from it.

LOG 075 · 20 January 2026 · Black hole

Intuition: I noted a heading about perspective inside a black hole and new time.

Outcome: OPEN. The note has only a heading; no equation, no measurement, nothing to test.

LOG 076 · Night of 1 September 2026 · The most important number

Intuition: When I thought all the way down to base 1, I sensed that the most important number might be 3 rather than 2. In this view I thought of 2 as something like 0.

Outcome: OPEN. I did not define a measure of “importance”; it cannot be tested.

Led to: → LOG 040 (the seed of Base^0)

LOG 077 · 2-3 September 2026 · Numbers are alive

Question: I was thinking of numbers as living, fluid things resembling acceleration, rather than fixed values. I asked myself whether I was building a derivative machine.

Outcome: OPEN. The philosophical part cannot be tested. The testable part—that every difference of Fibonacci is Fibonacci again—proved true.

Led to: → LOG 041

LOG 078 · 2-3 September 2026 · Screw and spring

Intuition: I pictured a spring extending and turning along an axis at the touch of an iron needle in a spring factory. I wondered whether such a movement could describe the behaviour of numbers.

Question: I considered the possibilities that the rotating thing was a screw, that it rose as it advanced, and that moduli related to the screw's direction.

Outcome: OPEN. The analogy has no measurable counterpart. The underlying bookkeeping—that the number of turns gives the quotient—is proved and known.

LOG 079 · 2-3 September 2026 · Repunit encryption

Intuition: I thought Mersenne primes might lose their importance and new encryption methods might turn to repunit primes.

Outcome: OPEN. There is no security analysis. The only testable part is that R_a(b) is composite for composite a; that is true.

LOG 080 · 3 September 2026 · Spring intuition and the screw question

Intuition: My first intuition here was not a screwdriver, but a movement resembling a spring: a line beginning at one point, extending and turning. Then I asked myself, "Is this not a screwdriver?" because if the rotating line rises as it advances, it can be thought of as a screw.

Research and test: I related the screw idea to the modulo operation. When dividing a number by 2^p−1, the notation x = q(2^p−1)+r gives q complete turns and the remainder r. Since 2^p ≡ 1 (mod 2^p−1), blocks of p base-2 places wrap to the same remainder in a Mersenne modulus; the quotient carries the number of turns and the remainder the position within the cycle. The FOLD step adds the high and low parts as N=(N>>p)+(N&M_p). This arithmetic relationship was tested; the screw analogy was not shown to be a physical or geometrical theorem.

Outcome: OPEN. Folding held; the three-dimensional spring idea has not yet been formulated.

LOG 081 · 7 September 2026 · Zoom

Intuition: I was thinking of everything through representations of 11; in this view I gave no place to 0. Rather than numbers growing, I imagined a movement like zooming or folding.

Outcome: OPEN. This is a perspective, not a falsifiable claim.

LOG 082 · 8 September 2026 · Paper folding

Question: I built an analogy in which I knew the number of folds, the paper's density and its weight. I wondered whether preservation or change of aspect ratio could say something about primality.

Outcome: FAILS. I had never defined the "number of folds"; when it was defined, every number was seen to reduce to 1 in the end.

LOG 083 · 15 September 2026 · All Mersennes are the same number

Intuition: I thought of all Mersenne numbers as the same number written with different instructions. I read [2,1,0]_3 through 3x2+1, interpreting the shift when 0 appeared as an instruction to multiply by 3.

Outcome: OPEN. I did not define whether “the same number” meant representation or value. Reading a digit as an instruction is a convention; it does not yield a claim to verify.

LOG 084 · 15 September 2026 · Odd and even

Intuition: I was considering redefining evenness through divisibility by the smallest prime in the number universe I was in; I objected to treating the odd/even distinction as given from the start. I read 22 not only as 2x11 but within a universe progressing as 2, 22, 222, 2222. From that viewpoint I thought 22 might also be the smallest prime.

Outcome: OPEN. I did not define “universe”. If defined as a base, the smallest prime in every universe is still 2.

LOG 085 · 15 September 2026 · Infinity

Intuition: I thought the infinitude of Mersenne primes might follow from this mechanism.

Note: I did not record which operation or chain of theorems I meant by “mechanism” here.

Outcome: OPEN. Whether there are infinitely many Mersenne primes is an open question; it cannot be settled by a finite test (T16_mersenne_infinitude).

LOG 086 · 21 September 2026 · Rubik

Intuition: I imagined that these structures could turn into an intersection or Venn diagram, or even a Rubik’s cube method.

Outcome: OPEN. A nice analogy; its rules have not yet been written, so how it would enter the processor has not been tested.

---

6. Sources

My own archive

The dates are those inside the files or their modification dates. The filenames are the names in my archive; they remain as records of their day.

- **20 January 2026** · My idea notebook (black hole note)
- **21 April 2026** · CT-CPU archive: CTCPU_BIBLE.txt, MERSENNE folder, mersenne_ctcpu_64xbit.txt
- **23 April 2026** · CT-CPU archive: journal_001_log_modu_k_karari.txt
- **April-July 2026** · CT-CPU archive: LEMKE_OLIVER_BIAS_DETAYLI.txt
- **8 June 2026** · CT-CPU journey summary; CT-CPU R&D journal, LOG-001 and LOG-002
- **9 June 2026** · CT-CPU archive: 02_golge_kesif_2026-06-09
- **June 2026** · CT-CPU archive: KISISEL_IFADE.txt
- **18 July 2026** · My manifesto
- **29 July 2026** · Hunter archive: EMULATOR_YOL_HARITASI.md
- **2 August 2026** · My idea notebook
- **5 August 2026** · Hunter archive: ARASTIRMA_050826_137_SAYI_EVRENI.txt

- **21 August 2026** · Hunter archive: ARASTIRMA_210826_DUNYA_VE_CTCPU.txt
- **30-31 August 2026** · My private journal: CEVAPLAR 3, 4, 5 and 6
- **1-3 September 2026** · My idea notebook (01.09, 03.09); conversation record (night of 1-2 September); 2-3 September session compilation
- **4-5 September 2026** · Conversation records (4 September 23:02; 5 September 00:48); TEST_001 experiment report
- **7 September 2026** · DEXMERSENNE archive: dm.txt, SAYI_TEST_ALGORITMASI_001.txt (04:07), PAKETLI MERSENNE ELEGI.txt (04:22); conversation records (03:55, 04:37); Hunter archive: HUNTER_PIPELINE_SONUCLARI_001.txt; my idea notebook
- **8-12 September 2026** · Conversation record (8 September 18:22); my idea notebook (08.09, 10.09, 12.09)
- **15 September 2026** · CxCPUMath archive: Reversed Mersenne P (08:02); my idea notebook, F39 (14:19); TEST_003 experiment report
- **19 September 2026** · Hunter archive: ARASTIRMA_20260919 (Ulam band); my idea notebook
- **21 and 30 September 2026** · Hunter archive: final form (Rubik)
- **1-2 October 2026** · Hunter archive: CEM_11_ILK_CIFT_011026.txt (1 October 00:57), HUNTERCPU_6X7_021026; my idea notebook (01.10)
- **10-11 October 2026** · Hunter theories (separate record); chronology of Hunter ideas, test notebook and Appendix A
- **11-12 October 2026** · Ladder working notes and Pascal −15…+15 drawing
- **13-14 October 2026** · Mersenne row tables (separate repository: pascal-mersenne-rows)
- **Undated** · Hunter archive: HUNTER.txt (the machine's current state), CEM_OKUMA_001_US_SAYACI.txt

The tests in this repository: journal_tests.c and its output TEST_OUTPUT.txt.

Literature

1. R. J. Lemke Oliver, K. Soundararajan, Unexpected biases in the distribution of consecutive primes, PNAS 113 (2016). https://doi.org/10.1073/pnas.1605366113
2. J. Clark Scott, But How Do It Know? The Basic Principles of Computers for Everyone (2009).
3. Gödel numbering. https://en.wikipedia.org/wiki/G%C3%B6del_numbering
4. Logarithmic number system. https://en.wikipedia.org/wiki/Logarithmic_number_system
5. Prime factorisation. https://mathworld.wolfram.com/PrimeFactorization.html
6. Mersenne numbers, Fermat and Euler. https://mathworld.wolfram.com/MersenneNumber.html
7. GIMPS, the mathematics of Mersenne primes. https://www.mersenne.org/various/math.php
8. Reduction modulo Mersenne numbers, CACR technical report CORR 99-39 (1999). https://cacr.uwaterloo.ca/techreports/1999/corr99-39.pdf
9. Multiplicative order. https://mathworld.wolfram.com/MultiplicativeOrder.html
10. P. L. Montgomery, Modular multiplication without trial division, Math. Comp. 44 (1985). https://en.wikipedia.org/wiki/Montgomery_modular_multiplication
11. Horner's method. https://mathworld.wolfram.com/HornersMethod.html
12. A. Avizienis, Signed-digit number representations for fast parallel arithmetic, IRE Trans. Electronic Computers (1961). https://doi.org/10.1109/TEC.1961.5219227
13. Carry-save adder. https://en.wikipedia.org/wiki/Carry-save_adder
14. Base. https://mathworld.wolfram.com/Base.html

15. Binomial theorem. https://mathworld.wolfram.com/BinomialTheorem.html
16. OEIS A001405, central binomial coefficients (step rule: C. Hanusa, 2003). https://oeis.org/A001405
17. OEIS A051920 (N. J. A. Sloane, 1999). https://oeis.org/A051920
18. OEIS A193242 (A. Povolotsky, 2013). https://oeis.org/A193242
19. OEIS A000108, Catalan numbers. https://oeis.org/A000108
20. OEIS A007051, (3^n + 1)/2. https://oeis.org/A007051
21. L. Jiu, C. Vignat, On binomial identities in arbitrary bases, J. Integer Sequences 19 (2016). https://arxiv.org/abs/1602.04149
22. M. S. Islam and colleagues, Generating binomial coefficients in a row of Pascal's triangle from extensions of powers of eleven, Heliyon (2022). https://arxiv.org/abs/2005.14490
23. Legendre's formula. https://en.wikipedia.org/wiki/Legendre%27s_formula
24. Double Mersenne numbers. https://mathworld.wolfram.com/DoubleMersenneNumber.html

7. Manifesto

My personal message is in a separate file: A Little Message.